Logistic Regression –

```
# Load libraries

import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix, precision_score, recall_score
import seaborn as sns
import matplotlib.pyplot as plt
```

Load Dataset

```
# Replace with your dataset path
df = pd.read_csv("Invistico_Airline.csv")

# Inspect first rows
df.head()
```

	satisfaction	Customer Type	Age	Type of Travel	Class	Flight Distance	Seat comfort	Departure/Arrival time convenient	Food and drink	Gate location
0	satisfied	Loyal Customer	65	Personal Travel	Eco	265	0	0	0	2
1	satisfied	Loyal Customer	47	Personal Travel	Business	2464	0	0	0	3
2	satisfied	Loyal Customer	15	Personal Travel	Eco	2138	0	0	0	3
3	satisfied	Loyal Customer	60	Personal Travel	Eco	623	0	0	0	3
4	satisfied	Loyal Customer	70	Personal Travel	Eco	354	0	0	0	3

5 rows × 22 columns

```
df.columns
df.shape
```

```
(129880, 22)
```

```
# Check target distribution
df['satisfaction'].value_counts(normalize=True)
```

satisfaction	proportion
satisfied	0.547328
dissatisfied	0.452672

dtype: float64

Encode Categorical Variables

```
# We need to convert categorical predictors into numeric format.

categorical_cols = df.select_dtypes(include=['object']).columns.drop('satisfaction')
df_encoded = pd.get_dummies(df, columns=categorical_cols, drop_first=True)
```

Train-Test Split

```
X = df_encoded.drop("satisfaction", axis=1)
y = df_encoded["satisfaction"].map({"dissatisfied":0, "satisfied":1})

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, stratify=y, random_state=42
)
```

Train Logistic Regression Model

```
# Impute missing values with the mean (calculated from X_train to avoid data leakage)
X_train_imputed = X_train.fillna(X_train.mean())
X_test_imputed = X_test.fillna(X_train.mean())

model = LogisticRegression(max_iter=1000)
model.fit(X_train_imputed, y_train)
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_logistic.py:465: ConvergenceWarning: lbfgs
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
  n_iter_i = _check_optimize_result(
```

LogisticRegression

```
LogisticRegression(max_iter=1000)
```

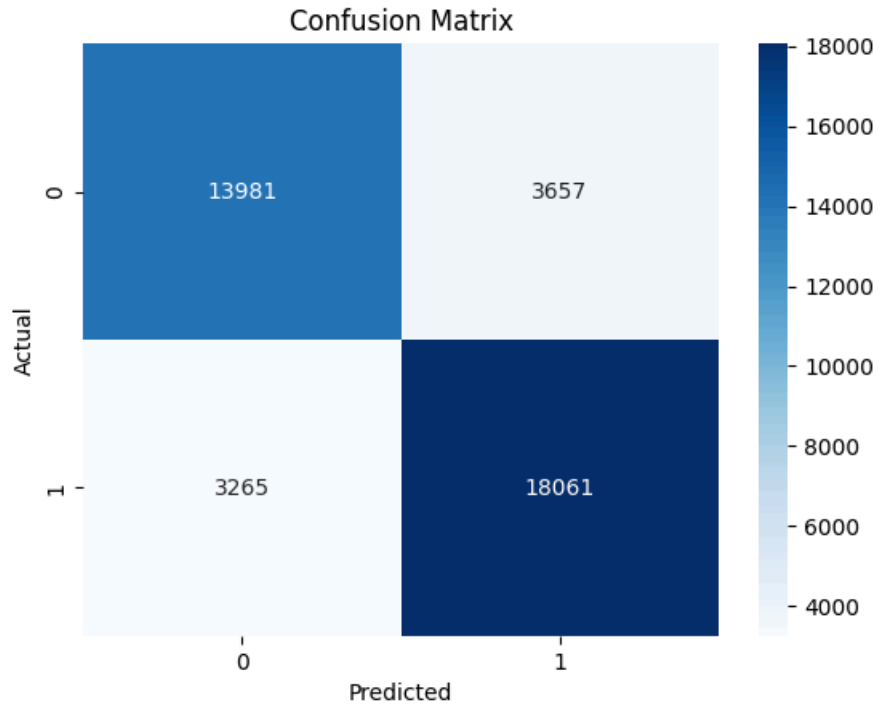
Model Evaluation

```
y_pred = model.predict(X_test_imputed)

# Confusion Matrix
```

```
cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()

print("Precision:", precision_score(y_test, y_pred))
print("Recall:", recall_score(y_test, y_pred))
```




```
Precision: 0.8316143291279123
Recall: 0.8469004970458596
```

Interpret Coefficients

```
coeffs = pd.DataFrame({
    "Feature": X.columns,
    "Coefficient": model.coef_[0]
}).sort_values(by="Coefficient", ascending=False)

coeffs.head(10)
```

	Feature	Coefficient
7	Inflight entertainment	0.604928
2	Seat comfort	0.333807
10	On-board service	0.299911
9	Ease of Online booking	0.279383
13	Checkin service	0.189163
11	Leg room service	0.183977
15	Online boarding	0.166831
5	Gate location	0.067605
8	Online support	0.053973
12	Baggage handling	0.043248

Next steps: Generate code with coeffs | New interactive sheet

Interpret coefficients and formulate recommendations

```
# Create dataframe of coefficients
coeffs = pd.DataFrame({
    "Feature": X.columns,
    "Coefficient": model.coef_[0]
}).sort_values(by="Coefficient", ascending=False)

# Display top positive and negative drivers
top_positive = coeffs.head(10)
top_negative = coeffs.tail(10)

print("Top Positive Drivers of Satisfaction:\n", top_positive)
print("\nTop Negative Drivers of Satisfaction:\n", top_negative)

# Map features to business recommendations
recommendations = {
    "DepartureDelay_in_minutes": "Invest in better scheduling and operations to reduce delays.",
    "Seat_comfort": "Redesign seating or offer affordable upgrades to improve comfort.",
    "Checkin_service": "Streamline digital check-in and train staff to reduce wait times.",
    "Inflight_wifi_service": "Upgrade connectivity for faster, more reliable in-flight internet.",
    "Food_and_drink": "Improve meal quality and variety to enhance passenger experience.",
    "Onboard_service": "Train crew for better customer interaction and service delivery.",
    "Class_Business": "Promote premium seating benefits to boost satisfaction among economy travel
}

print("\nBusiness Recommendations:")
for feature, action in recommendations.items():
    if feature in coeffs["Feature"].values:
        print(f"- {feature}: {action}")
```

```
Top Positive Drivers of Satisfaction:
                    Feature  Coefficient
7   Inflight entertainment     0.604928
2             Seat comfort     0.333807
10        On-board service     0.299911
9   Ease of Online booking     0.279383
13         Checkin service     0.189163
11        Leg room service     0.183977
```

```
15         Online boarding     0.166831
5            Gate location     0.067605
8           Online support     0.053973
12        Baggage handling     0.043248

Top Negative Drivers of Satisfaction:
                               Feature  Coefficient
17           Arrival Delay in Minutes    -0.006986
14                       Cleanliness    -0.007515
0                                Age    -0.020878
4                      Food and drink    -0.159892
6               Inflight wifi service    -0.184147
3   Departure/Arrival time convenient    -0.278497
19    Type of Travel_Personal Travel    -0.696876
20                         Class_Eco    -1.059893
21                     Class_Eco Plus    -1.337523
18   Customer Type_disloyal Customer    -2.407870

Business Recommendations:
```